Registry Redesign — Architecture Review

Status: Proposal **Branch:** `docs/registry-redesign-architecture` **Date:** 2026-04-20 **Inputs:** `docs/planning/registry-redesign/README.md` (D1–D7), `docs/design.md`, `CLAUDE.md`, `backend/species/*`, `frontend/components/*`, `frontend/app/*`

This document resolves the six open questions posed by the planning hub. Decisions D1–D7 are treated as locked-in constraints; see the **Concerns** section at the end for anything I think deserves a second look.

---

1. Migration audit — design.md §4 Species shape vs. `backend/species/models.py`

The design doc §4 enumerates fields the prototype UI reads. Below is the full delta against the current `Species` model, each tagged G1-G (Gate 1 graceful-degrade — frontend hides when absent), G2 (Gate 2 migration — new field), M (already on the model in some form), or D (drop — not worth adding).

design.md field	current state	category	Notes
`id`	`id` int PK	M	No change.
`scientific_name`	`CharField(200)`	M	
`provisional_name`	`CharField(100) null`	M	Already present; profile UI must render `"{genus} sp. {provisional_name}"` for undescribed morphospecies — front-end convention already in `SpeciesCard.tsx`.
`common_names` (string[])	`CommonName` FK table with `is_preferred` flag	M (shape differs)	Already modeled richer than the design doc assumed. Serializers already inline `common_names`. No migration needed.
`family`	`CharField(100)`	M	String, not FK. Stays string in Gate 1 (D4 — add, don't change). A future `Family` model is explicitly out of scope.
`genus`	`CharField(100)`	**→ FK**	Being promoted; see Q2 below. This is the single exception to "no schema changes in Gate 1" called out in D1.
`authority`	`CharField(200) null`	M	
`year_described`	`IntegerField null`	M	

`taxonomic_status`	`CharField` choices	M	Choices already include `described`, `undescribed_morphospecies`, plus two more.
`endemic_status`	`CharField` choices	M	
`iucn_status`	`CharField` choices (mirror)	M	Denormalized mirror; never write directly.
`iucn_criteria`	**absent** on Species; lives on `ConservationAssessment.criteria`	**G1-G**	Profile hero can optionally pull most-recent-accepted assessment's criteria for display; if not wired in Gate 1, hide the criteria string. No migration.
`cares_status`	`CharField` choices (`CCR`, `CEN`, `CVU`, `CLC`)	M (value set differs)	Design doc used `CCR / priority / monitored / ""`. Current model uses the actual CARES four-tier vocabulary. **Keep current vocabulary** (it is the correct one). Front-end filter chips must adopt the real values; the design doc is wrong here. Noted under Concerns.
`shoal_priority`	`BooleanField`	M	
`max_length_cm`	`DecimalField(5,1) null`	M	
`habitat` (one line)	`habitat_type: CharField(100)`	M	Close enough; `habitat_type` is the one-liner the card needs.
`basin` (canonical watershed name on Species)	**absent** on Species. Basins live on `SpeciesLocality.drainage_basin` (FK to `Watershed`).	**G1-G**	Design doc assumed a denormalized "primary basin" on Species. We do not denormalize. Card metadata row ("Family · Endemic · Basin") shows a derived value: the drainage basin of the species' primary locality, or omit when none. Serializer adds a computed `primary_basin` string field in Gate 1 (derived, not stored) or the card hides the basin slot. Prefer: add read-only `primary_basin` to

			`SpeciesListSerializer`, no migration.
`description`	`TextField`	M	
`has_husbandry`	serializer-computed	M	Already a serializer method.
`has_localities`	serializer-computed	M	Already a serializer method.
`localities` (count)	**absent** as scalar	**G1-G**	Add a `locality_count` serializer method (cheap; it's already an `exists()` check — promote to `count()`). No migration.
`ex_situ.institutions / individuals / programs`	`ex_situ_summary` already on detail serializer	M	Already computed. Gate 1 wires card/profile to read from this.
`silhouette_url`	`silhouette_svg` (inline SVG text)	M (shape differs)	Already solved, better than doc's `silhouette_url`. Inline SVG + `SpeciesSilhouette.tsx` path stands.
`hero_photo_url`	**absent**	**G2**	Profile hero §16.4 depends on this. Gate 1: render the stripe-pattern fallback on profile hero for every species. No photos shipped in Gate 1.
`hero_photo_credit`	**absent**	**G2**	Ships with hero_photo_url.
`silhouette_credit`	**absent**	**G2** (optional)	We can author into `silhouette_svg` `<title>` / `<desc>` for Gate 1; dedicated credit field can wait.

Summary for Gate 1: zero new fields on `Species`. Everything above either exists, is computed in a serializer, or degrades gracefully on the front end. The one schema change in Gate 1 is the `genus` FK promotion (Q2), plus two new models (`Genus`, `SiteMapAsset`). This is well within the "add, don't change" rule.

Gate 2 fields confirmed: `hero_photo_url`, `hero_photo_credit`, plus the larger list in README D6 (`provisional_name` — already here; `taxonomic_status` — already here; `endemic_status` — already here; `cares_status` — already here; `shoal_priority` — already here). The only genuinely new fields in Gate 2 are the two hero-photo fields plus whatever the `FieldProgram` / `ExSituHolding` expansion adds.

---

2. Genus model shape

Why promote at all

Decision D1 requires a genus-level silhouette fallback. `Species.genus` is currently a string — there is no place to hang a `silhouette_svg` against "the genus Paretroplus". A new `Genus` model is the minimum structural change.

We are **not** promoting `Species.family` to a FK. Family fallback is explicitly rejected. Family stays a string.

Proposed model

New app file: `backend/species/models.py` (same module).

```
class Genus(models.Model):
    name = CharField(max_length=100, unique=True)     # e.g. "Paretroplus"
    family = CharField(max_length=100)                # denormalized string, same
vocabulary as Species.family
    silhouette_svg = TextField(blank=True,
        help_text="Inline SVG used as fallback when no species-level silhouette is
set.
                   Same authoring rules as Species.silhouette_svg. Rendered at ~300px
wide.")
    silhouette_credit = CharField(max_length=300, blank=True)     # optional
    notes = TextField(blank=True)                                  # curator notes,
admin-only
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)

    class Meta:
        db_table = "species_genus"
        verbose_name_plural = "genera"
        ordering = ["name"]

    def save(...):
        # Same strip_svg_root_size_attrs logic as Species.save (share via module-level
helper).
```

Then on `Species`:

```
genus = models.CharField(max_length=100)                # retained for one release
(read-through)
genus_fk = models.ForeignKey(Genus, on_delete=PROTECT,
                             null=True, blank=True,
                             related_name="species",
                             db_column="genus_id")
```

Keep the string column during the Gate 1 transition (D4: add, don't change). We can drop `Species.genus` (the string) in a Gate 2 cleanup migration once everything reads from the FK. Inventing a property that hides the two-field transition would complicate the mirror-policy thinking; the string plus the FK side-by-side is easier to reason about and uses ~100 extra bytes per row.

Migration strategy (three-step)

Migration A — schemaonly, adds the model and the nullable FK.

- `CreateModel Genus` (fields as above).

- `AddField Species.genus_fk` (nullable FK, no default, no data migration yet).
- Admin surfaces `Genus` on the admin index.

Migration B — data migration (Python, RunPython, reversible).

- For every distinct value of `Species.genus` (the string), create a `Genus` row if none exists. `family` on the new row is copied from the most-common `Species.family` paired with that genus string (we have one-to-one genus→family integrity today; a sanity assertion raises if not).
- For every `Species`, set `genus_fk_id` to the matching `Genus.pk`.
- Reversal: null out `genus_fk`, delete `Genus` rows that have zero incoming species.

Migration C — tighten constraint.

- Alter `Species.genus_fk` to `null=False`.
- Add DB-level index on `genus_fk_id` (should be automatic via ForeignKey).
- Do not drop the `genus` string column yet. That is Gate 2 cleanup, and only after all read paths have been migrated.

Admin surface

`@admin.register(Genus)` with:

- `list_display = ["name", "family", "species_count", "has_silhouette"]`
- `search_fields = ["name"]`
- `list_filter = ["family"]`
- `readonly_fields = ["created_at", "updated_at"]`
- Fieldset 1 "Identity": `name`, `family`
- Fieldset 2 "Silhouette": `silhouette_svg`, `silhouette_credit` (help text identical to Species — the authoring conventions are the same)
- Fieldset 3 "Curator notes": `notes`

`SpeciesAdmin` gains `raw_id_fields = [..., "genus_fk"]` (preserving autocomplete UX) and its `list_filter` gains `"genus_fk"`. The existing `genus` string field is rendered read-only once Migration C lands.

Ripple through serializers / filters / views

- `SpeciesListSerializer` — replace `"genus"` source with `genus_fk.name` (or keep `"genus"` field name; add a `SerializerMethodField` that prefers the FK and falls back to the string during the transition window).
- `SpeciesDetailSerializer` — same. Additionally, add a nested `genus` object: `{"name": str, "has_silhouette": bool}` so the frontend knows whether the genus fallback SVG is available **without** shipping the SVG body on every list response. The full genus SVG is fetched by the frontend only when rendering a species that has no species-level SVG.
- `SpeciesFilter` (django-filter) — add a `genus` filter using `field_name="genus_fk__name"`. Retain backward compat with the string filter for one release.
- `SpeciesViewSet.get_queryset` — add `.select_related("genus_fk")` to avoid N+1s.
- `views_dashboard.py` / `views_map.py` — audit for uses of `species.genus` string; swap to `species.genus_fk.name` once migration B runs.

Frontend silhouette resolution (three-step)

`SpeciesSilhouette.tsx` currently: species SVG or nothing.

New contract:

```
resolve(species):
  if species.silhouette_svg:        return species SVG          # render existing
figure
  if species.genus?.silhouette_svg: return genus SVG            # same figure chrome,
different SVG
  return null                                                   # design-explicit:
render nothing
```

The genus SVG body is fetched in the profile-page server component via a small API endpoint `GET /api/v1/genera/<name>/silhouette/` (public, cached, text/html-ish payload) **only when** `species.silhouette_svg` is empty. Do not embed every genus SVG in list-page payloads.

For the directory list, the three-step fallback is **not** needed in Gate 1 — the card silhouette slot is optional and can render nothing when species SVG is absent. Adding genus-fetch to list responses is a perf regression we shouldn't take on.

No family fallback. No generic silhouette. Nothing, per D1.

---

3. Map asset model — `SiteMapAsset`

The redesign calls for admin-uploaded PNG thumbnails at specific page slots (D2). Proposed model:

```
class SiteMapAsset(models.Model):
    class Slot(models.TextChoices):
        HERO_THUMB    = "hero_thumb",    "Home hero — ~160×320, small map beside Red
List breakdown"
        PROFILE_PANEL = "profile_panel", "Species profile — ~180×360, distribution
panel thumbnail"
        # Future slots register here. Each choice is ONE row, not many.

    slot          = CharField(max_length=40, choices=Slot.choices, unique=True)
    image         = ImageField(upload_to="site-maps/")
    alt_text      = CharField(max_length=300,
                              help_text="Screen-reader description of what the map
shows.")
    credit        = CharField(max_length=300, blank=True)
    expected_width_px  = PositiveIntegerField()          # rendered as admin hint; not
enforced
    expected_height_px = PositiveIntegerField()
    usage_notes   = TextField(blank=True,
        help_text="Where this image renders on the site and under what conditions.
                   E.g. 'Shown on the home page hero when the live map widget is
                   not interactive. Never displayed larger than 180×360.'")
    updated_at    = DateTimeField(auto_now=True)

    class Meta:
        db_table = "species_sitemapasset"
        ordering = ["slot"]
```

One row per slot (the `unique=True` enforces it). If an admin uploads a new file for an existing slot, they edit the existing row rather than create a duplicate.

Admin surface

`@admin.register(SiteMapAsset)` with:

- `list_display = ["slot", "image_preview", "updated_at"]`
- A read-only `image_preview` method that renders a `<img>` tag in admin (≤320px wide).
- Help text on every field. The `slot` select shows both the machine name and the human description directly (TextChoices labels already provide this).
- Pre-seed via data migration: two empty rows for `hero_thumb` and `profile_panel` so admins are editing known slots rather than inventing names.

Frontend consumption

Add a small server-side helper: `lib/siteMapAssets.ts`

```
export async function getSiteMapAsset(slot: "hero_thumb" | "profile_panel") {
  // cached fetch of /api/v1/site-map-assets/<slot>/  or embedded at build time via
  // dashboard payload, whichever keeps the home page simple
  // returns {url, alt, credit, width, height} or null
}
```

Components reference by slot, not by URL:

- `HomeHero` calls `getSiteMapAsset("hero_thumb")`, renders `<img>` if present, falls back to a sunken-background stripe pattern if absent (per design §16.4 hero-when-no-photo treatment, applied to the map slot when no admin asset is uploaded yet).
- `SpeciesProfile` distribution panel calls `getSiteMapAsset("profile_panel")` for the thumbnail; interactive MapClient renders below it.

Public endpoint `GET /api/v1/site-map-assets/<slot>/` → `{url, alt, credit, width, height}` or 404. Cached aggressively; invalidate on admin save via the existing `revalidate_public_pages` hook (see `species/admin_revalidate.py`).

---

4. Gate 1 component migration order

Recommendation: **tokens → chrome → cards → pages, horizontal layers**, not vertical slices.

Why not vertical slices. A vertical-slice approach (do the Home page fully, then Directory, then Profile) would leave the site visually mixed — half in the old styles, half in the new — for most of the gate window. The Home and Directory pages share the card, the filter chips, the IUCN pill, and the site header. Porting them twice is wasted effort.

Order (and the stopping points where partial completion still ships something coherent):

1. **Tokens + type** (1–2 days). Promote `docs/design.md` §12 + §13 into `globals.css` + a Tailwind theme extension or raw CSS custom properties. Load Spectral + IBM Plex Sans + IBM Plex Mono. Swap `--accent`, `--ink-*`, etc. Every existing component consuming Tailwind's slate/sky palette now inherits the journal palette via variable alias. **Stop-ship checkpoint A: site is visually re-skinned everywhere, functionally identical.** This alone would ship coherently.

2. **Chrome** (1–2 days). `SiteHeader`, `SiteFooter`, nav active-pill treatment, footer grid, last-sync strip. These are the parts the eye lands on first and are entirely standalone. **Stop-ship checkpoint B: header/footer match the design, pages still use old card styling.** Acceptable if we run out of time here.

3. **Primitives** (2–3 days). IUCN pill (§15.1), basin pill (§15.7), filter chips (§15.3), segmented control (§15.4), buttons (§15.6), inputs (§15.5). Replace `IucnBadge.tsx` internals, add new `BasinPill.tsx`, `FilterChip.tsx`, `SegmentedControl.tsx`. These are consumed by cards and page layouts.
4. **Silhouette cascade** (1 day backend + 1 day frontend). Ship the `Genus` model + admin + API endpoint; update `SpeciesSilhouette.tsx` to fall through species → genus → null. This work is independent of visual token migration and can land in parallel with steps 1–3.
5. **Map assets** (0.5 day backend + 0.5 day frontend). `SiteMapAsset` model + admin + endpoint. Home hero + profile panel start reading from it. Gracefully absent while content team uploads.
6. **Cards** (2 days). Rebuild `SpeciesCard.tsx` to the §15.2 spec (left color bar, silhouette column, metadata row, right IUCN pill + softened CARES/SHOAL). Apply to Directory and any home-page silhouette grid.
7. **Pages** (3–5 days, in this order): a. Home (`frontend/app/page.tsx`) — stat hero, Red List breakdown histogram, three-up cards, last-sync strip. b. Species Directory (`frontend/app/species/page.tsx`) — filter rail + card grid + density control (density as a GET param, no client state needed). c. Species Profile (`frontend/app/species/[id]/page.tsx`) — hero strip (no hero photo in Gate 1 — stripe fallback), three-up meta strip, sections per §16.4. d. Map page (`frontend/app/map/page.tsx`) — split layout (`MapListView.tsx` already exists; re-skin rather than replace). e. Dashboard — visual re-skin only in Gate 1; full dashboard reorientation is Gate 3 per D6.

Partial-completion outcomes

- If we only finish through step 3 (primitives) — site is re-skinned, pills and chrome look right, cards still old. Acceptable for demo. Ship.
- If we finish through step 6 (cards) — Directory and Home read correctly; Profile and Map are re-skinned but layout is old. Ship.
- If we finish step 7 — Gate 1 complete.

---

5. Route scope for Gate 1

Per D3: **existing URLs are authoritative, no `/registry/*` move**. The mapping from design-doc routes to production routes:

design.md	production (Gate 1)	treatment
`/registry` (home)	`/`	Full redesign. Stat hero + three-up + sync strip.
`/registry/species`	`/species/`	Full redesign. Filter rail + card grid + density selector.
`/registry/species/{slug}`	`/species/{id}/`	Full redesign of layout; **slug routing is Gate 2+** — keep numeric IDs. Adding `scientific_name`-slug aliases is not in scope for June 1.
`/registry/map`	`/map/`	Visual re-skin only. Split layout per §16.5 if time allows; otherwise preserve the current layout with new tokens. MapClient internals untouched (D2).

`/registry/dashboard`	`/dashboard/`	**Visual re-skin only.** The "demote Red List snapshot, lift ex-situ coordinator panels" reorg is Gate 3.
`/registry/about`	`/about/` (+ `/about/data`, `/about/glossary`)	Visual re-skin (tokens, type, chrome). No structural changes.

Deferred to later gates:

- `/contribute/husbandry/*` — keep as-is, visual re-skin only. Contribution flows were not touched by the design handoff.
- `/species/{id}/husbandry` — husbandry detail stays on its own style unless the visual tokens bleed in automatically (they will, via `globals.css`).
- Slug aliases for species URLs.

No URL renames. No redirects. New routes not introduced in Gate 1.

---

6. Accessibility baseline delta

What's already in place

- `IucnBadge.tsx` uses `aria-label` and includes the two-letter code as text (§10, §19.2 compliant).
- `SpeciesCard.tsx` wraps in `<Link>` (keyboard navigable); IUCN pill has `aria-label`.
- `MapClient.tsx` has `aria-label` on `MapContainer` describing count and purpose, and emits an `sr-only` summary (`data-testid="map-summary"`). Legend uses `aria-expanded`.
- `SpeciesSilhouette.tsx` renders within a `<figure aria-label={altText}>`.
- `SpeciesFilters.tsx` (need to audit, but presumed) uses labeled form controls.

Gaps against design §10 + §19

1. **Focus ring** — Currently relies on Tailwind's `focus-visible` defaults (ring-sky-500). Spec requires `2px solid var(--focus)` with 2px offset. One-file change in `globals.css` once tokens are in.
2. **Hit targets on filter chips and map legend toggles** — Design requires min 44px. Current chip/toggle buttons need audit; probably underfilled. Part of primitives rebuild (step 3 above).
3. **Color-alone status signaling** — IUCN pills already include the code, fine. Need to verify CARES/SHOAL soft labels (§7) include `aria-label`-style text; today `SpeciesCard.tsx` renders them as plain text, which is already code-inclusive. Still OK once redesign lands.
4. **Map dot keyboard focus + screen reader** — Leaflet `CircleMarker` is not in the keyboard tab order by default. The prototype spec requires "Paretroplus menarambo, Bemarivo basin, Critically Endangered" reachable by keyboard. This is a known Leaflet a11y gap; options: a. Add a keyboard-navigable list view (already exists: `MapListView.tsx`) and treat the map as a secondary, visual view. Defensible; matches §16.5 split layout. b. Add `role="button"` + `tabIndex=0` shims to markers. Leaflet-specific hackery. Don't. Recommendation: lean on `MapListView.tsx` for the keyboard path; document the map as visual-complement in a11y statement.
5. **Contrast** — `--ink-3 (#6B7670) on --bg-raised (#FFFFFF)` fails AA for small body text (§19 explicitly notes this). Enforce in code review: ink-3 reserved for ≥12px captions, not body.
6. **Silhouette alt text** — Currently "Illustrative silhouette of {scientific_name}" via `aria-label` on the figure. Design says `alt={species.scientific_name}` on the img. Our `<figure aria-label>` is equivalent-or-better. No change required.

7. **Route transition animation (§18)** — 200ms fade + 4px rise on main container only. Must honor `prefers-reduced-motion`. Add the reduced-motion query guard when implementing.
8. **Skip-to-content link** — Design doc doesn't mention it but WCAG baseline requires it and we don't seem to have one. Add during chrome step.

Gate 1 a11y work is entirely cosmetic (focus ring, hit targets, reduced-motion guard, skip link). No structural a11y refactor needed — the existing components were built to a reasonable baseline.

Risks & open questions

R1 — `Genus.family` can drift from `Species.family`

The new `Genus` model carries a denormalized `family` string. If a species has its `family` string edited in admin, the matching `Genus.family` won't auto-update. Mitigation options: (a) make `Species.family` a computed property that reads from `Genus.family` once the FK migration completes, (b) add a data-integrity test that asserts 1:1 genus→family. Picking (b) for Gate 1; (a) is a Gate 2 candidate.

R2 — `locality_count` on every list row is a potential N+1

Fixable with `.annotate(locality_count=Count("localities"))` on the list queryset. Flagging so the PM spec's acceptance criteria include this on list performance.

R3 — Pre-seeding `SiteMapAsset` rows

Data migration needs to create empty `hero_thumb` and `profile_panel` rows so admins can edit rather than create. Without pre-seeding, admins may create duplicate/mis-spelled slot names. Pre-seeding the two Gate 1 slots is in scope.

R4 — `iucn_status=NE` vs. `NULL` — already solved

Noted because design.md §4 assumed a non-null `iucn_status` column with an "NE" sentinel. We use `NULL`. `SpeciesFilter.filter_iucn_status` already reconciles this; no further work.

R5 — Dashboard gets a visual skin only in Gate 1

The design doc §16.6 assumes the full ex-situ-coordinator reorg landed. D6 defers that to Gate 3. The architecture proposal here keeps the Gate 1 dashboard visually in the same shape as today. PM spec should scope dashboard AC tightly.

Concerns (things I want to flag, not silently work around)

1. **CARES vocabulary mismatch.** Design doc uses `CCR / priority / monitored`. Current model uses the real CARES four-tier codes `CCR / CEN / CVU / CLC`. The model is right. Front-end filter chips must adopt these. Sync copy with `@conservation-writer` when writing the filter labels (e.g. "CARES Critical", "CARES Endangered"). The `shoal_priority` boolean is fine as-is.
2. **Slug URLs for species.** The design doc assumes `{slug}` URLs and a slug column. We don't have one, and D4 says don't add. Gate 1 keeps numeric IDs. If marketing / SEO surface asks for slugs, treat as a Gate 2+ decision with a redirect table.
3. **No family FK.** D1 forbids family fallback, so `Species.family` stays a string. But this means filter-by-family in the admin and API still goes against a string column, and a species-with-typo'd-family is possible. Low impact given the ~6 families and curator discipline, but worth a pre-seed list validator at some point.

4. **Hero photo fallback** for species without `hero_photo_url` (Gate 1 reality for every species). Design §16.4 says "stripe pattern if absent." We must actually build that fallback; it's easy to forget and leave every profile with a blank hero strip.
5. **Genus silhouette authoring budget.** Alex's current authoring effort targets species-level silhouettes. Gate 1 ships the cascade but may have zero genus SVGs populated. That's fine — the cascade degrades to "nothing" and matches D1. Make sure the content plan doesn't assume genus coverage before Gate 1 demo.

Hand-off to downstream agents

This architecture proposal does not modify `.claude/agents/*`. The project-specific context those agents need (Domain Model, User Roles, Feature Areas, etc.) is already covered by `CLAUDE.md` and the BA/PM docs for prior gates. The PM agent writing the Gate 1 spec should pull acceptance criteria from:

- Q1 migration table (no new Species fields → AC: migrations touch only Genus, SiteMapAsset, and nullable Species.genus_fk).
- Q2 three-step migration plan (AC: schema migration reversible; data migration idempotent; post-migration assertion that every Species has a non-null genus_fk).
- Q3 one-row-per-slot unique constraint + pre-seeded rows (AC).
- Q4 ordered work plan (AC: stop-ship checkpoints A/B/C each produce a visually coherent site).
- Q5 route list (AC: no route renames, no redirects).
- Q6 a11y punch-list (AC: focus ring tokenized, 44px hit targets on chips/toggles, reduced-motion guard, skip link).

The BA agent reviewing whether to scope additional Gate 1 items should treat this doc's G1-G fields (criteria, primary_basin, locality_count) as free wins — they are serializer-only changes — versus G2 fields (hero_photo_url, hero_photo_credit) which need migrations and curator content and should stay deferred.